

```

#pragma once
#include <string>
#include <utility>
#include <vector>
#include <iterator>
#include <string.h>

/*
Part 1: Build Your Own Linked List
Implement and test a linked list that supports adding, searching, removing, and printing items. Do
not use std::list.
**Made adding the new nodes
**Printed new nodes
**Made the remove node

Part 2: Monopoly Game
Use your linked list to build a circular board with at least 10 properties.
**Made my linked list to a doubly linked list
**Made atleast 10 properties

Each property stores its name, cost, owner, and next pointer.
**Made all of them

Players move around the board, wrapping back to the beginning.
**Made it loop around

Players can buy unowned properties, but cannot buy owned properties.
**Can buy empty properties but not owned

Simulate at least 10 turns and print the results

BONUS POINTS IF YOU CREATE A GUI 😊
**Created a GUI assisted with QT
Briefly explain the time complexity of adding, searching, removing, moving, and traversing.

*/

//*****
//*****
//%%%%%%%%%% WE ARE GOING TO MAKE THIS WHOLE FILE OUR BOARD THAT INCLUDES PROPERTY
//%%%%%%%%%%
//*****
//*****

//Creates my linked list with a default head null pointer if empty
struct Property{
    std::string name;
    int cost;
    int owner = -1;
    Property* next = nullptr;
    Property* prev = nullptr;

    Property(std::string name, int cost) : name(std::move(name)), cost(cost){}
};

class Board {
private:
    Property* head = nullptr;
    Property* tail = nullptr;
    int count = 0;

```

```

public:
    Board() = default;
    Board(const Board&) = delete;
    Board& operator = (const Board&) = delete;
    ~Board() {clear();}

    //Make a set function that adds a new node
    void append(const std::string& name, int cost){
        //Adds a new node to the heap
        Property* node = new Property(name, cost);

        //If empty make a the new node as head
        if(head == nullptr){
            head = tail = node;
            node->next = node;
            node->prev = node;
        }
        //Else if not empty, make the new node be sent to the tail
        else{
            tail->next = node;
            node->prev = tail;
            node->next = head;
            head->prev = node;
            tail = node;
        }
        ++count;
    }
    //Use this to remove values, will search along the array to do so
    bool remove(const std::string& name){
        Property* node = search(name);
        if(!node){
            return false;
        }

        if(count == 1){
            head = tail = nullptr;
        }

        else{
            node->prev->next = node->next;
            //Assigns previous to point to the previous node
            node->next->prev = node->prev;

            //If our temp value is head point the new head next
            if(node == head){
                head = node->next;
            }
            //If our temp value is tail, point to the previous node.
            if(node == tail){
                tail = node->prev;
            }
        }
        delete node;
        --count;
        return true;
    }

    //Use this to search for a value and print it
    Property* search(const std::string& name) const {
        if(!head){
            return nullptr;
        }

        Property* current = head;
    }

```

```

    do{
        if(current->name == name){
            return current;
        }
        current = current->next;
    }while(current != head);
    return nullptr;
}

//Using this to move arround the board
Property* move(Property* from, int steps, bool* passedStart = nullptr) const{
    if(passedStart){
        *passedStart = false;
    }
    Property* current = from;

    for(int i = 0; i < steps; i++){
        current = current->next;
        if(current == head && passedStart){
            *passedStart = true;
        }
    }
    return current;
}

//Move to Certain space
std::vector<Property*> toVector() const{
    std::vector<Property*> out;
    if(!head){
        return out;
    }

    Property* current = head;

    do{
        out.push_back(current);
        current = current->next;
    }while(current != head);

    return out;
}

Property* start() const {
    return head;
}

int size() const{
    return count;
}

void clear(){
    if(!head) return;
    tail->next = nullptr;
    Property* current = head;

    while(current){
        Property* next = current->next;
        delete current;
        current = next;
    }
    head = tail = nullptr;
    count = 0;
}

};

```